ChatBot

Retrieval-Augmented Generation (RAG) Chatbot for NIT Jamshedpur

Submitted By

Abhinav Dev C (2023UGCS099)

Chandrima Hazra (2023UGCS017)

Hemanvitha Pullela (2023UGCS001)

Om Raj (2023UGCS022)

Pritom Roy (2023UGCS044)

Supervised By

Dr. Koushlendra Kumar Singh

Assistant Professor

Department of Computer Science & Engineering

National Institute of Technology Jamshedpur

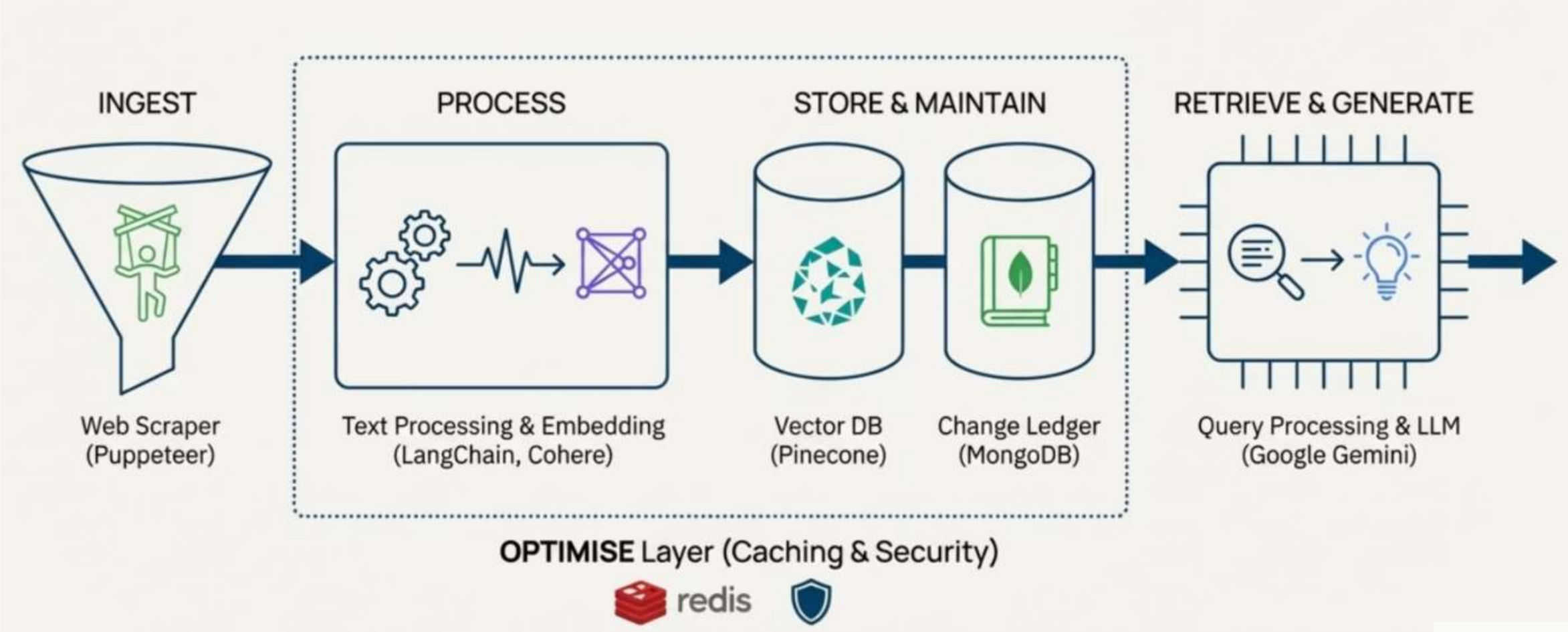
Background & Motivation

- Institutional websites like NIT Jamshedpur contain a **huge amount of information spread across pages, PDFs, notices, and dynamic updates**, making it difficult for users to quickly find what they need.
- Users—students, faculty, and applicants—often face **confusion due to fragmented content and limited keyword-based search**, which cannot understand intent or connect related information.
- With the website constantly evolving, **manual tracking of updates becomes inefficient**, leading to outdated knowledge and frequent queries to administration.
- Modern advances in **NLP, LLMs, and RAG frameworks** now enable more intelligent systems that retrieve relevant data in real-time and generate accurate, context-aware responses.
- A RAG-powered chatbot provides a **centralized and intuitive interface**, simplifying access to academic, administrative, and institutional information while reducing administrative workload.

This solution ultimately aims to **enhance information accessibility, reduce user frustration, and strengthen the overall digital experience** at NIT Jamshedpur.

System Architecture Overview

Our system architecture follows the complete journey of information—from **raw website data** to **accurate, context-aware answers**—through a **modular pipeline** that handles **ingestion, processing, storage, maintenance, and retrieval**. Web content is **scraped**, converted into **semantic embeddings**, and stored in a **vector database**, while a **change-tracking ledger** keeps the knowledge base up to date. During queries, the system retrieves the **most relevant context** and uses an **LLM** to generate **precise, grounded responses**, all supported by **caching and security layers** for optimal performance.



1. Advanced Scraping Pipeline for Dynamic and Static Data

Challenge: Modern websites are not static. Content is often loaded dynamically via JavaScript, and crucial information is locked in PDF documents.

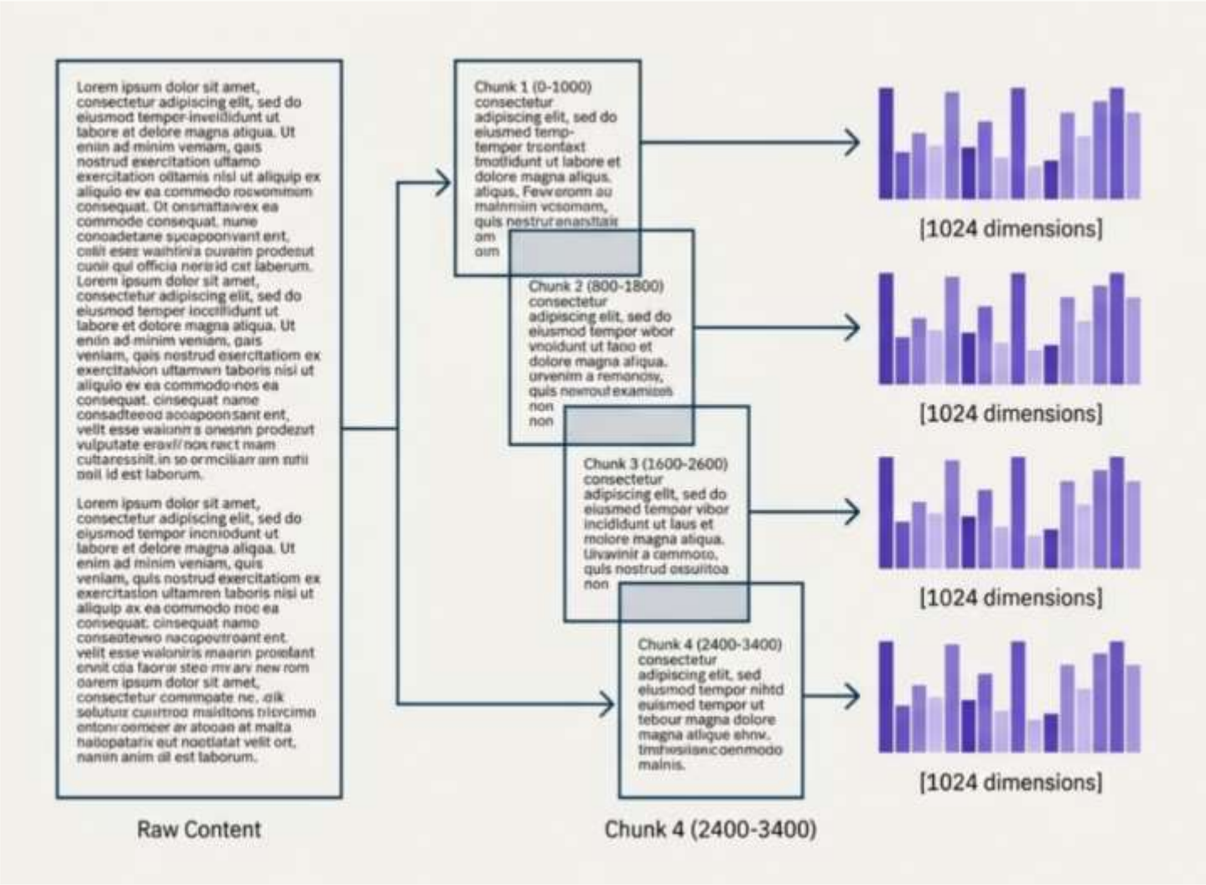
```
// Puppeteer waits for network activity to cease
const page = await browser.newPage();
await page.goto(url, { waitUntil: 'networkidle2'
});

// Wait for a specific dynamic element to render
await page.waitForSelector('.content-loaded');

// Extract text content after JS execution
const content = await page.evaluate(() =>
   document.body.innerText);
```

Solution: Our Puppeteer-based scraper implements an intelligent and comprehensive crawling strategy:

- **Sitemap-Aware Discovery:** Ensures systematic and complete coverage of all accessible pages.
- **Dynamic Content Handling:** Executes JavaScript and intercepts XHR/fetch requests to capture asynchronously loaded information such as recent notices and updates.
- **PDF Link Extraction and Processing:** Identifies, collects, and processes linked PDFs for dedicated text extraction and inclusion in the knowledge base.
- **Auto-Refresh & Change Detection:** Periodically scans the website to detect new or modified content, ensuring the knowledge base stays continuously updated.

2. Converting Raw Text into High-Dimensional Semantic Embeddings

Challenge: Modern websites are not static. Content is often loaded dynamically via JavaScript, and crucial information is locked in PDF documents.



Text Chunking Strategy: We use LangChain's *RecursiveCharacterTextSplitter* to break the scraped text into meaningful segments while preserving context and embedding quality.

- **Chunk Size:** 1000 characters to maintain coherence.
- **Chunk Overlap:** 200 characters to ensure smooth continuity between chunks.

Embedding Generation: Each chunk is transformed into a high-dimensional semantic vector capturing its meaning.

- **Model:** Cohere's *embed-english-v3.0* embedding model.
- **Vector Size:** 1024-dimensional embeddings.
- **Process:** Includes text normalisation, batch embedding, and a Redis-based caching layer to reduce redundant embedding requests and improve efficiency.

3. Efficient Knowledge Refresh with Change Ledger Management

Challenge: Institutional websites change constantly. Re-processing the entire site daily is computationally expensive and inefficient.

Solution: We use MongoDB as a lightweight **change-tracking ledger** to detect updates efficiently.

- `Pages` **Collection:** Stores each page's URL, content hash, and last-scraped timestamp.
- `Chunks` **Collection:** Tracks chunk hashes, parent page mapping, and Pinecone vector IDs.

Update Workflow: During each re-scrape, new hashes are compared with the ledger.

- **Unchanged pages** are skipped to save compute.
- **Modified or new pages** are re-chunked, re-embedded, and updated in Pinecone.
- **Deleted content** triggers removal of stale vectors, keeping the knowledge base clean and current.

```
// Pages Collection
{
  "_id": "ObjectId('...')",
  "url": "nitjsr.ac.in/admissions",
  "contentHash": "sha256:a3c5...",
  "lastScraped": "2025-11-16T10:00:00Z"
}
```

```
// Chunks Collection
{
  "_id": "ObjectId('...')",
  "page_id": "ObjectId('...')",
  "chunkHash": "sha256:f9b2...",
  "vectorId": "pinecone_vec_12345"
}
```

4. Vector-Based Query Retrieval for Precise Context Matching

Challenge: User queries can be ambiguous or phrased differently from the stored content, making it difficult to consistently retrieve the most relevant chunks. Ensuring accurate semantic matching requires reliable embeddings and effective similarity search to avoid irrelevant or missing context.

Query Processing: The user's question is embedded using the same Cohere embedding model, generating a 1024-dimensional query vector that aligns with the vector representations stored in the knowledge base.

Retrieval: Pinecone performs a cosine-similarity search to fetch the top-K (typically 5–10) most semantically relevant text chunks, ensuring that only the most meaningful context is selected.

Context Construction: The retrieved chunks are assembled into a structured prompt for the LLM, including system instructions, source-linked context, the user's query, and strict guidelines to ensure the model produces factual, grounded, and properly cited responses.

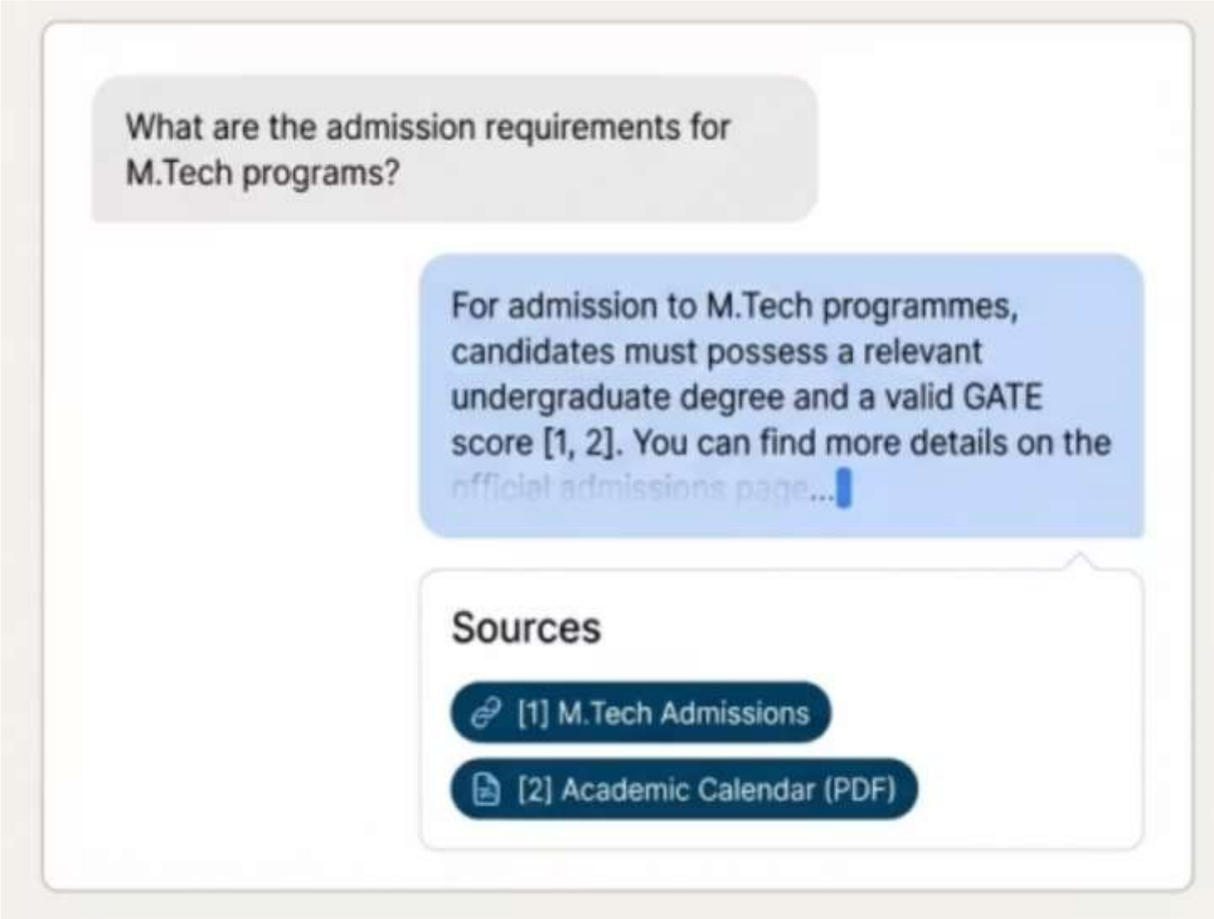
Knowledge Base Context:
[1] The M.Tech admission requirements include...
(Source: nitjsr.ac.in/admissions/mtech/span)
[2] All applicants must possess a valid GATE score...
(Source: nitjsr.ac.in/admissions/mtech/span)
...

Current Question: What are the admission requirements for M.Tech programs?

Instructions:
- Base your answer primarily on the context from the database.
- If context lacks information, clearly state that.
- For web pages, say: "See [Page Title]: [URL]"

5. LLM-Powered Response Generation with Source-Backed Reasoning

Challenge: Modern websites are not static. Content is often loaded dynamically via JavaScript, and crucial information is locked in PDF documents.



Generation Engine: We use Google's `gemini-2.5-flash` model to synthesise a comprehensive and coherent response from the retrieved context, ensuring the final output is accurate and well-structured.

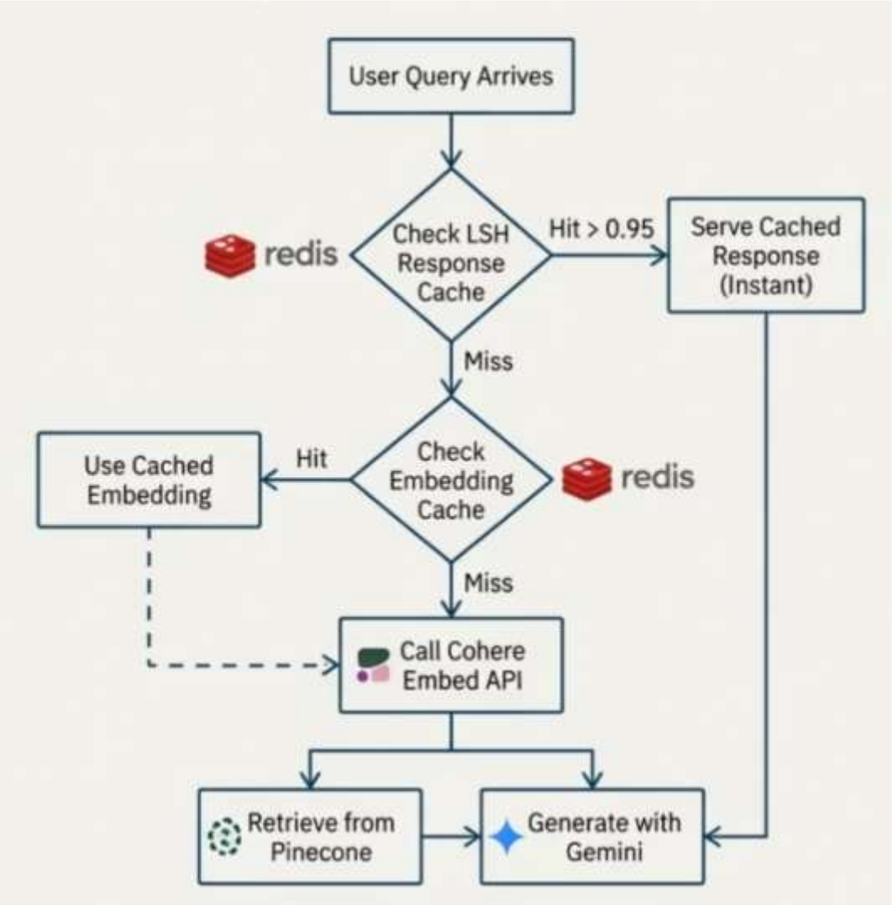
Streaming Responses: The system uses Server-Sent Events (SSE) to stream the answer token-by-token. This provides instant user feedback, significantly reducing perceived latency and making the interaction feel more responsive.

Source Attribution: Every response is grounded in verifiable facts. The final answer includes clear citations and direct links to the original source documents, enabling users to validate information and explore it further with confidence.

A multi-layer caching strategy dramatically improves performance and manages API costs.

Layer 1: Embedding Cache (Redis-backed LRU)

- **Function:** Stores precomputed embeddings using text hashes as keys to allow quick reuse.
- **Impact:** Eliminates redundant Cohere embedding API calls for repeated text chunks or user queries, significantly reducing latency and computation. A 24-hour TTL ensures freshness while maintaining efficiency.



Layer 2: Semantic Response Cache (LSH-based)

- **Function:** Utilises Locality-Sensitive Hashing (LSH) to detect semantically similar queries based on vector similarity.
- **Impact:** If an incoming query embedding has a cosine similarity greater than 0.95 to a cached query, the stored response is served instantly—bypassing Pinecone retrieval and Gemini generation. This mechanism reduces overall API usage by nearly 80%.

The system delivers significant, measurable improvements in response time and efficiency.

Rigorous testing demonstrates the system's success across key performance indicators. The caching layers provide a >90% improvement in average response time.

Table 1: Average Response Times (s)

Query Type	Without Cache	With Cache	Improvement
Simple Factual	4.0s	0.3s	75%
Complex Multi-part	6.2s	0.5s	92%
Average	**5.33s**	**0.40s**	**92.5%**

Table 2: Answer Accuracy Metrics

Metric	
Metric	Score
Correct Answers	80%
Partially Correct	10%
Source Attribution Accuracy	75%

Table 3: Cache Performance

Cache Type	Hit Rate
Cache Type	Hit Rate
Embedding Cache	78%
Response Cache (LSH)	42%

The system shows strong improvements in both speed and quality. Caching and optimized retrieval reduce the average response time from **5.33s to 0.40s** (≈92% faster). Answer accuracy remains high, with **80% correct responses** and **75% source attribution**. Embedding and LSH caches achieve **78%** and **42%** hit rates, significantly lowering API usage.Overall, the system delivers faster, more reliable, and cost-efficient performance.

Comprehensive admin tools and a REST API enable system monitoring and management.

The system includes secure, JWT-authenticated administrative endpoints for system health monitoring, manual operational triggers, and performance analysis. A web-based dashboard provides at-a-glance statistics on vector counts, page counts, and cache performance.

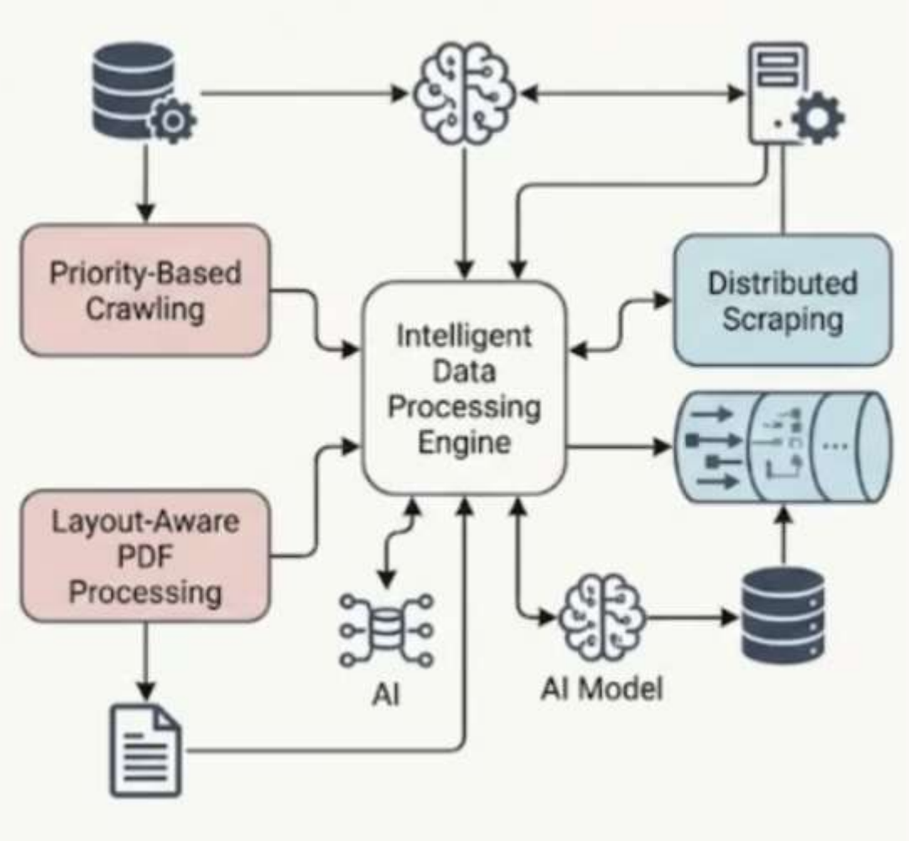
API Endpoints

Endpoint	Description
POST /scrape-and-embed	Combined scraping and embedding operation
POST /chat-stream	Streaming chat endpoint using SSE
GET /health	System health check and database status
GET /stats	Aggregate statistics (vector count, cache hits)

Future work will focus on deeper integration and more intelligent data processing

1. Scraper Logic Enhancements

- **Priority-Based Crawling:**Increase the crawling frequency for highly dynamic sections—such as notices and announcements—while scraping static pages less often to optimise system resources.
- **Layout-Aware PDF Processing:**Implement smarter extraction algorithms to accurately read multi-column PDFs, tables, and complex layouts, ensuring precise text retrieval from academic documents.



2. Backend and Architecture Improvements

- **Distributed Scraping:**Run multiple scraper instances in parallel to reduce full-site crawl times from ~60 minutes to under 10 minutes, dramatically improving update freshness.
- **Direct Database Integration:**Integrate directly with the university's internal management systems to fetch real-time data on course schedules, exam timetables, faculty information, and library resources.

Thank You